GUÍA DEL TALLER · UNIDAD 2

Mi primera feature guiada por spec

Programa de Capacitación · **Desarrollando con IA**

80 minutos | Individual | Día 1 · bloque tarde

EL CAMINO QUE VAS A RECORRER

Historia vaga del PO → **spec.md** tus decisiones → **plan.md** aprobado → **Tests** en verde → **Commit** revisado

01 · EL CASO

Tu misión

Primer sprint del **portal web de Pedidos**. Los vendedores necesitan consultar pedidos desde el navegador, sin entrar al sistema Delphi. Tu repo ya tiene la base: una **API en ASP.NET Core** con **45 pedidos de ejemplo en memoria**. Lo que falta es el listado con filtros.

LA HISTORIA QUE DEJÓ EL PRODUCT OWNER

> *«Como vendedor, quiero listar pedidos filtrando por cliente, estado y rango de fechas.»*

LO QUE LA HISTORIA NO DICE

- ¿Qué **estados** existen?
- ¿Y si las **fechas** vienen invertidas?
- ¿Cuántos **resultados** por página?
- ¿Cómo se informa un **error**?

Si le entregas la historia así al agente, **inventará las reglas por su cuenta**. Hoy haces de PO y de Dev: tú decides y lo escribes en la spec.

TU RUTA DE 80 MINUTOS

1 · 10'	2 · Spec · 15'	3 · Plan · 15'	4 · Código + tests · 25'	5 · 10'	6

1 Prompt maestro 2 Completar la spec 3 Plan mode 4 Implementar con tests 5 Revisar el diff 6 Cierre

Antes de empezar, ten listo

- Tu **pedidos-web** actualizado con la base de la U2 (README · «Cómo empezar», camino A)
- Rider abierto en tu repo **pedidos-web**
- **.NET 10 SDK** (`dotnet --version`)
- **Claude Code 2.1.277 o superior** (`claude --version`)
- Leíste **agents.md** (3 min, en el receso)
- El **prompt maestro**: repo de material, `unidad-2/taller/prompt-maestro.md`

02 · LO JUSTO PARA TRABAJAR

Ideas que vas a usar

La cadena SDD (Spec-Driven Development)

spec.md
El **qué**: reglas y criterios. Lo decides tú.

→

plan.md
El **cómo**: lo propone el agente, lo apruebas tú.

→

Código + tests
Un test por criterio de aceptación.

Plan mode

Claude explora y propone un plan, pero **no edita tus archivos** hasta que apruebes. Ábrelo directo con `claude --permission-mode plan` o, dentro de la sesión, con `Shift+Tab` hasta ver **plan mode on**.

AGENTS.md

Las reglas del proyecto: comandos, convenciones y reglas de trabajo. **Viene vacío: lo escribes tú en el paso 1. No ejecutes** `/init`: crea un CLAUDE.md y Claude dejaría de leer tu AGENTS.md.

Un criterio de aceptación que se puede probar

Formato **Dado / Cuando / Entonces**, con los datos de ejemplo del repo y un resultado exacto:

Dado	los 45 pedidos de ejemplo del repositorio
Cuando	filtro por el cliente C-004
Entonces	recibo exactamente 5 pedidos (ids 7, 15, 23, 31 y 39), todos de C-004

Regla de oro: si no sabes cómo escribir el test de un criterio, el criterio todavía no está bien escrito.

¿Quieres ver un ejemplo completo? En el repo de material, `unidad-2/demos/tiendamini` (el caso visto en clase) tiene una spec real con 14 criterios y su trazabilidad hasta los tests.

PASO 1 · PARTE 1 DE 2

El prompt maestro

10 min

Claude explora tu repo, te entrevista y escribe dos borradores: tu **AGENTS.md** y tu **spec.md**. Te ahorra el formato, no las decisiones.

ANTES DEL PASO 1 · EN EL RECESO

Lee la portada de agents.md (3 min)

Markdown normal
Sin campos obligatorios. Lo leen varios agentes: Claude Code, Codex, Copilot y otros.

Secciones sugeridas
Proyecto, comandos para compilar y testear, estilo de código, tests y seguridad. Opcional: commits y pull requests.

Lo que no se adivina
Se escribe lo que el agente no puede deducir. Lo obvio y lo que no es verdad en el repo sobra.

¿Vienes de la Unidad 1? Tus reglas anteriores siguen en el historial de Git: `git log -p -- AGENTS.md`. Rescata las que te sirvan.

CÓMO USARLO · 4 PASOS

1 · Abre en plan
En la terminal de Rider, dentro de tu `pedidos-web`:

```
claude
--permission-mode plan
```

La barra muestra ⏸ **plan mode on**: Claude no edita nada.

2 · Pega
Copia el prompt completo (página siguiente) desde `unidad-2/taller/prompt-maestro.md`. Copiar desde el PDF puede cortar líneas.

3 · Responde
Claude te hace hasta 6 preguntas, de a una: 2 sobre cómo quieres que trabaje y el resto sobre la historia. Responde con frases cortas.

4 · Escribe
`Shift+Tab` hasta ver ⏸ **manual mode on** y pide: *«Escribe los dos archivos»*. Claude te pide permiso por cada archivo: **léelo antes de aceptar**.

⚠ **¿Por qué modo manual en el 4?** Al salir de plan mode con un solo `Shift+Tab` caes en ⏩ **auto mode on**, y ahí Claude escribe sin preguntarte. Sigue presionando hasta ver ⏸ **manual mode on**.

💡 El prompt te ahorra el formato, no las decisiones. Claude dejará marcados como `TODO` los puntos que decides tú en el paso 2.

PASO 1 · PARTE 2 DE 2

El prompt, para copiar

PROMPT · COPIA Y PEGA EN CLAUDE CODE

```
Actúa como analista técnico de un equipo Scrum. Soy el Product Owner y el desarrollador
de esta historia, y quiero aplicar Spec-Driven Development.

CONTEXTO DEL PROYECTO
- Repositorio: portal web "Pedidos". API REST en ASP.NET Core (.NET 10) con tests xUnit.
- Los datos son de ejemplo y viven en memoria (no hay base de datos).
- El AGENTS.md está vacío a propósito: lo vamos a escribir juntos.
- Historia de usuario del backlog:
  "Como vendedor, quiero listar pedidos filtrando por cliente, estado y rango de fechas."

TU TAREA, EN ESTE ORDEN
1. Explora el repositorio SIN modificar nada: estructura, proyectos, modelos, datos de
   ejemplo, endpoints y tests existentes. Anota los comandos y convenciones que puedas
   comprobar en los archivos.
2. Hazme una entrevista de máximo 6 preguntas, de a una por vez. Las 2 primeras sobre cómo
   quiero que trabajes en este repo (por ejemplo, qué puedes hacer sin preguntarme).
   El resto, sobre la historia. Pregunta solo lo que el código no responde.
3. Con lo que encontraste y con mis respuestas, genera DOS borradores:

   a) AGENTS.md (máximo 40 líneas, en español), con las secciones que recomienda agents.md:
      - Proyecto: qué es, en 2 líneas.
      - Comandos: compilar, testear y ejecutar la API. Solo los que comprobaste en el repo.
      - Estilo de código: convenciones detectadas (idioma de los nombres, tipo para el
        dinero, tipo para las fechas, qué va en Core y qué va en Api).
      - Tests: framework, dónde viven, cómo se nombran y cuándo se ejecutan.
      - Seguridad: nunca secretos ni claves en el código.
      - Reglas de trabajo. Incluye siempre estas tres:
        "toda funcionalidad parte de una spec aprobada en specs/<historia>/spec.md",
        "un test xUnit por criterio de aceptación",
        "no hacer commit sin tests en verde".
        Agrega solo las que salgan de mis respuestas.

   b) specs/HU-01-listar-pedidos/spec.md, con estas secciones:
      - Historia de usuario
      - Objetivo (1 párrafo)
      - Reglas de negocio
      - Criterios de aceptación (checklist verificable, con los datos de ejemplo del repo)
      - Ejemplos de entrada y salida
      - Errores esperados
      - Fuera de alcance
      - Preguntas abiertas

REGLAS QUE NO PUEDES ROMPER
- NO inventes reglas de negocio ni reglas de trabajo. Si una decisión no salió de mis
  respuestas, escribe "TODO: decidir ..." y explica en una línea por qué importa.
- En AGENTS.md no escribas comandos ni convenciones que no hayas visto en el repo.
- En la spec, deja como TODO, como mínimo: la lista de estados válidos, qué pasa con
  fechas invertidas, el tamaño de página y el formato de error.
- NO escribas código ni tests. NO crees plan.md: eso lo haremos después en plan mode.
- Al final, muéstrame la lista de TODO pendientes de los dos archivos.
```

Cópialo desde el repo de material: `unidad-2/taller/prompt-maestro.md`. Ahí también está el checklist para revisar los dos borradores.

PASO 2 · EL CORAZÓN DEL TALLER

Completa la spec tú

15 min

Abre `specs/HU-01-listar-pedidos/spec.md` y resuelve **cada TODO** con una decisión tuya. Estas cuatro son obligatorias:

Estados válidos

¿Qué estados puede tener un pedido? ¿Qué pasa si filtran por uno que no existe?

Fechas invertidas

Si «desde» es posterior a «hasta», ¿error o lista vacía? ¿Las fechas son inclusivas?

Paginación

¿Tamaño por defecto? ¿Máximo? ¿Qué devuelve la respuesta además de los pedidos?

Formato de error

¿Qué código HTTP? ¿Qué cuerpo? Ej.: `{ "codigo": "...", "mensaje": "..." }`

ESCRIBE AL MENOS 5 CRITERIOS · CRITERIO FLOJO V/S CRITERIO SÓLIDO

Flojo (no se puede probar)	Sólido (ya es casi un test)
«Las fechas se validan correctamente.»	**Dado** desde 2026-10-31 y hasta 2026-10-01, **cuando** listo pedidos, **entonces** recibo 400 con `codigo` FECHAS_INVERTIDAS.
«La lista se pagina.»	**Dado** los 45 pedidos de ejemplo, **cuando** pido la página 3 con tamaño 20, **entonces** recibo 5 pedidos y total 45.

Tu checklist antes de seguir

AGENTS.md

- [ ] Lo leí y corregí lo que no es verdad en este repo
- [ ] Probé sus comandos (`dotnet build`, `dotnet test`)
- [ ] Tiene las 3 reglas SDD y las reglas de trabajo que **yo** decidí
- [ ] Al abrir `claude` aparece una línea como «AGENTS.md loaded»

spec.md

- [ ] No queda ningún `TODO`
- [ ] Tengo 5 o más criterios y cada uno se puede probar
- [ ] Hay 2 o más ejemplos de entrada y salida, uno de error
- [ ] «Fuera de alcance» dice qué **no** se construye

Punto de control

El instructor revisa un criterio por fila. Pregunta clave: **¿otra persona podría escribir el test sin preguntarte nada?**

PASO 3

Del qué al cómo: plan mode

15 min

Con la spec lista, vuelve a plan mode (`Shift+Tab` hasta ver ⏸ **plan mode on**) y pídele a Claude un plan. Tu trabajo es **cuestionarlo** antes de aprobarlo.

PROMPT · EN PLAN MODE

```
Lee @specs/HU-01-listar-pedidos/spec.md y el AGENTS.md.
Propón un plan para implementarla: enfoque, archivos afectados,
riesgos y pasos. Incluye un test xUnit por cada criterio de aceptación.
No escribas código todavía.
```

Preguntas para cuestionar el plan

- ¿Cumple **cada regla** de mi spec? ¿Inventó alguna?
- ¿Pide al usuario datos que debería calcular el servidor?
- ¿Respeta las convenciones de mi AGENTS.md?
- **¿Hay un test por criterio?**
- ¿Los tests prueban la lógica en Core, no el endpoint?
- ¿Toca archivos que no debía?
- ¿Respeta lo que dejé «fuera de alcance»?

ASÍ SE VE UNA BUENA CORRECCIÓN

El agente propone
«Si las fechas vienen invertidas, las intercambio y sigo.»

↓

Tu revisión
«No. Mi spec dice error FECHAS_INVERTIDAS (400). Corrige el plan y agrega su test.»

CUANDO LO APRUEBES

```
Guarda el plan aprobado en specs/HU-01-listar-pedidos/plan.md
```

Corregir el plan cuesta minutos; corregir el código, horas. En el ejemplo de TiendaMini, el dev encontró **2 errores graves** solo leyendo el plan.

PASO 4

Implementa con tests

25 min

Con el plan aprobado, presiona `Shift+Tab` hasta ver ⏩ **accept edits on**: Claude edita sin preguntarte, pero te pide permiso antes de ejecutar comandos. El control fuerte viene en el paso 5.

PROMPT · IMPLEMENTACIÓN

```
Implementa el plan aprobado de @specs/HU-01-listar-pedidos/plan.md
de a un paso. Para cada criterio de aceptación escribe su test xUnit
con el ID del criterio en un comentario (// CA-01). Prueba la lógica
en PedidosWeb.Core; el endpoint solo traduce a HTTP. Ejecuta dotnet test
al terminar cada paso y muéstrame el resultado.
```

ASÍ SE VE UN TEST CON SU CRITERIO

C# · XUNIT

```
[Fact] // CA-03
public void Listar_FechasInvertidas_DevuelveError()
{
    // Arrange: desde > hasta
    // Act: llamar al servicio de Core (no al endpoint)
    // Assert: código de error EXACTO de tu spec
    Assert.Equal("FECHAS_INVERTIDAS",
        error.Codigo);
}
```

Adapta los nombres a las clases de tu repo. Lo importante: el **ID del criterio** en un comentario y un **Assert exacto**.

COMANDOS QUE USARÁS

TERMINAL

```
dotnet build
dotnet test
dotnet test --filter
  "FullyQualifiedName~Pedidos"
dotnet run --project
  src/PedidosWeb.Api
```

Nombres de test

`Metodo_Escenario_Resultado`

Ej.: `Listar_FechasInvertidas_DevuelveError`

TU MINI TABLA DE TRAZABILIDAD

CA	Qué verifica	Test xUnit	Estado
CA-01	…	…	✓ verde
CA-02	…	…	…

¿Quieres probarlo a mano? Con la API arriba (`dotnet run`), abre en el navegador la ruta que definiste en tu spec en `http://localhost:5080`. Los tests HTTP automáticos quedan como extensión.

PASOS 5 Y 6

Tú firmas

10 + 5 min

Antes del commit, revisa el diff completo con estas **5 preguntas**. Si no leíste el diff, no hay commit.

1. ¿Cumple la spec, criterio por criterio?
2. ¿Los tests prueban algo real? Rompe el código a propósito: ¿fallan?
3. ¿Hay casos borde sin cubrir?
4. ¿Hay secretos en el código?

5. ¿Cambió algo que no pedí? Si Claude implementó algo que no está en la spec, es un **hallazgo de revisión**, no un regalo: quítalo o agrégalo primero a la spec.

COMMIT

TERMINAL

```
git add AGENTS.md specs/ src/ tests/
git commit -m "feat: HU-01 listar pedidos con filtros
(spec + tests)"
git push
```

Entregable

- `AGENTS.md` escrito por ti
- `spec.md` sin TODO
- `plan.md` aprobado
- Código y tests en verde
- Commit revisado

Cierre (paso 6): tres personas muestran un criterio de su spec y el test que lo prueba. Elige uno de error, no solo el caso feliz.

PLAN B Y EXTENSIÓN

Te atrasaste

Usa la **carpeta** `unidad-2/taller/u2-respaldo` del repo de material: copia su `spec.md` a tu `specs/HU-01-listar-pedidos/`. Los comandos están en su README.

Falla la red

Revisa con la checklist el diff que proyecta el instructor.

Terminaste antes

Agrega un criterio nuevo y verifica que Claude lo implemente con su test. O suma tests HTTP del endpoint (requieren un paquete NuGet: decídelo en tu spec y en tu AGENTS.md).

03 · SI ALGO NO SALE

Errores frecuentes

Síntoma	Prueba esto
Al abrir `claude` la barra dice ⏩ **auto mode on**	Es el modo inicial desde la versión 2.1.283. Sal y abre con `claude --permission-mode plan`, o presiona `Shift+Tab` hasta ver ⏸ **plan mode on**.
Claude empezó a escribir código durante la entrevista	Presiona `Esc`, vuelve a plan mode y pega de nuevo el prompt.
Claude escribió archivos sin pedirte permiso	Estabas en auto mode. Revisa con `git diff` y deshaz lo que no corresponda. Para escribir los borradores usa ⏸ **manual mode on**.
La spec o el AGENTS.md traen reglas que tú no decidiste	Pide: *«Marca como TODO toda regla que no salió de mis respuestas.»*
Claude no sigue tu AGENTS.md	Ejecuta `/memory`. Si aparece un CLAUDE.md, renómbralo y abre una sesión nueva. Si recién actualizaste Claude Code, la primera sesión puede no leerlo: abre otra.
`dotnet test` no encuentra tests	Corre `dotnet build` y revisa que el proyecto de tests esté en la solución.
Todos los tests pasan, incluso con código roto	Tus Assert son débiles (`NotNull`, `True`). Compara el resultado **exacto** de tu spec.
El plan toca cosas que no pediste	Completa «Fuera de alcance» en la spec y vuelve a pedir el plan.
La sesión está lenta o confusa	`/context` para ver el uso; `/compact` para resumir sin perder el hilo.

🔁 **Una historia, una sesión.** Cuando termines la HU-01 y pases a otra cosa, usa `/clear`.

⚠ **Nunca** ejecutes `/init` en este repo: crea un CLAUDE.md que desactiva tu AGENTS.md.

04 · PARA APRENDER LO NECESARIO

Fuentes recomendadas

Si te trabas, estas fuentes te dan justo lo que necesitas. Todos los enlaces están también en el sitio del curso.

PASOS 1 A 3 · SPEC Y PLAN

Estándar AGENTS.md
Qué va en el archivo de reglas del proyecto.
agents.md

Claude Code · Memoria y AGENTS.md
Qué instrucciones lee Claude Code y cuándo.
code.claude.com/docs/en/memory

Claude Code · Permission modes
Plan mode, modo manual y accept edits.
code.claude.com/docs/en/permission-modes

AI-native SDLC playbook · Plan mode
Cómo pedir, cuestionar y aprobar un plan.
academy.claude.com/courses/ai-native-sdlc-playbook/plan-mode

Plantilla de spec comentada y ejemplo TiendaMini
Repo de material: unidad-2/demos/tiendamini
github.com/ffelipecuevasc/material-curso-ia-desarrolladores

PASOS 4 Y 5 · CÓDIGO Y REVISIÓN

Claude Code 101
El flujo explore → plan → code → commit.
anthropic.skilljar.com/claude-code-101

Best practices for Claude Code
Criterios de verificación y manejo del contexto.
code.claude.com/docs/en/best-practices

Common workflows · Work with tests
Pedir tests útiles y casos borde.
code.claude.com/docs/en/common-workflows

Testing en .NET con xUnit
[Fact], [Theory] y dotnet test.
learn.microsoft.com/dotnet/core/testing/unit-testing-csharp-with-xunit

Para ir más allá: Martinelli, *Spec-Driven Development: From Specs to Code with AI Agents* (Apress, 2026) y Osmani, *Beyond Vibe Coding* (O'Reilly, 2025).